

# 11 — Risk scoring engine

**What it does:** turns every stored check result into a single **explainable** 0–100 score, a tier ( `low` / `medium` / `high` ) and a list of human-readable reasons. It is a **transparent, weighted rules engine — not a model**. Every point in a score traces to a named rule and a sentence a reviewer can read.

**Code:** `services/risk_scoring_service.py` ( `score_case` , `evaluate_rules` , `pipeline_status` ), `tasks/risk_scoring_task.py` ( `score_case_task` ), `services/risk_rule_seed.py` (the default rules and `seed_risk_rules` ), `services/risk_rule_catalog.py` (what a rule editor may build), `api/settings.py` (rule and threshold administration), `api/platform.py` (the platform rule template), models `risk_rules` , `risk_settings` , `risk_rule_templates` , `case_risk_assessments` .

**Rules and thresholds are per company.** Each company has its own `risk_rules` and `risk_settings` , edited by its Reviewer L2s under *Settings > Risk Rules*; a case is always scored with its own company's rules. A new company starts from the platform's **rule template** (see [Rule templates](#)).

## When a case is scored

Every pipeline task ends with `request_case_scoring(case_id)` , which enqueues `score_case_task` on `forensics_queue` (and never raises if the broker is down). `score_case(db, company_id, case_id)` then:

1. **Gate — `pipeline_status(case)`** must be `complete` . For every document: extraction terminal; for every `complete` document: `field_validation` and `issuer_verification` terminal; for every **PDF**: `metadata_forensics` , `error_level_analysis` , `copy_move_detection` , `duplicate_detection` , `visual_inconsistency_review` terminal ( `completed` or `failed` ); and, when the case has  $\geq 2$  documents, a `cross_document_check_completed` event **newer than the newest upload**. Anything missing is listed in `pipeline.pending` and shown to the reviewer. ( `signature_stamp_detection` is intentionally **not** required — see data-flow.) Scoring a half-checked case would under-report and could let it be approved.
2. **Gather evidence:** the **latest completed** check per (document, check\_type), the case's `cross_document_findings` , and all `signature_matches` .
3. **Evaluate every current rule of the case's company** (highest `version` per `rule_id` , inactive included but never fired).
4. **Fingerprint** = SHA-256 of the sorted document ids + the sorted ( `rule_id` , `document_id` , `count` ) of fired rules. If it equals the latest assessment's `evidence_fingerprint` , **stop** — no new row.
5. **Score:** `raw_score` =  $\sum \text{weight}$  of fired rules, except that the **metadata rules ( `metadata.*` ) together add at most the company's metadata cap** ( `risk_settings.metadata_score_cap` , default **40**, editable next to the thresholds; 100 = no cap) — one edit leaves several metadata traces (modified after created, the edit history, a date after the file's creation, the editor's name), which must not max the score on their own. A cap that applied is recorded in the snapshot as `group_caps`: `{"metadata": {"cap", "points"}}` and shown on the case and in the report. `score` = `clamp(round(raw_score), 0, 100)`; `tier` from the company's thresholds ( `score`  $\geq$  `high`  $\rightarrow$  `high` ,  $\geq$  `medium`  $\rightarrow$  `medium` , else `low` ; defaults **30 / 60**).
6. **INSERT** a `case_risk_assessments` row (one per (case, fingerprint) — a unique constraint, so a redelivered task can't insert it twice), set `cases.risk_tier` , write `risk_assessment_completed` , and — if the case is still `submitted` / `under_automated_review` — transition it to `pending_manual_review` via the workflow service.

The fingerprint deliberately excludes weights, so a re-trigger can never quietly re-score an old case under newly tuned weights: a case is only re-scored when its *evidence* changes.

## Rules

A rule is a row in the company's `risk_rules` — configuration, not code:

| Column                                                                                                  | Meaning                                                                                                   |
|---------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>rule_id</code>                                                                                    | Stable key, <code>family.name</code> (e.g. <code>metadata.editing_software_detected</code> )              |
| <code>category</code>                                                                                   | <code>forensics</code> , <code>consistency</code> , <code>verification</code> or <code>duplication</code> |
| <code>check_type</code>                                                                                 | Which check's output it reads (used for display/grouping)                                                 |
| <code>condition</code>                                                                                  | JSON: <b>what to match</b> (below) — created by the catalog, not hand-written                             |
| <code>weight</code>                                                                                     | Points added when it fires (–100...100)                                                                   |
| <code>severity</code>                                                                                   | <code>info</code> / <code>low</code> / <code>medium</code> / <code>high</code> — shown beside the reason  |
| <code>reason_template</code>                                                                            | Sentence with <code>{placeholders}</code> the reviewer reads                                              |
| <code>is_active</code>                                                                                  | Off = evaluated but never fires                                                                           |
| <code>version</code> , <code>effective_from</code> , <code>updated_by</code> , <code>change_note</code> | Versioning trail                                                                                          |

### Condition kinds ( `condition.match` )

| Kind                         | Matches                                                                                                                                                                                 | Fires                                                                       |
|------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <code>finding</code>         | Findings in a check's <code>details</code> <b>list</b> whose <code>finding.name</code> equals <code>condition.finding</code> and (optionally) whose severity ∈ <code>severity_in</code> | <b>Once per document</b> , <code>count</code> = number of matching findings |
| <code>check_result</code>    | The check's overall <code>result</code> equals <code>condition.result</code> (e.g. <code>flag</code> )                                                                                  | Once per document                                                           |
| <code>sub_check</code>       | One sub-check in a <b>dict</b> -shaped <code>details</code> (field validation) with <code>status</code> = <code>flag</code>                                                             | Once per document                                                           |
| <code>cross_document</code>  | A <code>cross_document_findings</code> row for <code>condition.field_name</code> , optionally filtered by severity                                                                      | <b>Once per case</b> , <code>count</code> = matching findings               |
| <code>signature_match</code> | A <code>signature_matches</code> row whose <code>result</code> equals <code>condition.result</code>                                                                                     | Once per target document                                                    |

A rule fires **once per hit, regardless of how many findings a document has** — five separate `orphaned_objects` findings on one document add the weight once, not five times.

### Reason templates

`render_reason` fills the template from the hit's context. Available placeholders: `{document}` (filename), `{count}`, `{page}`, `{description}`, `{reason}`, `{issuer}`, `{person}`, `{field}`, plus any key in a finding's `data`. A placeholder with no value renders as an **empty string** (never an error). The **rendered** text is what is stored and shown.

### The default rules (63)

These are the rules in the platform template, i.e. what a new company starts with. Full tables of `rule_id`, condition and default weight are in each stage's doc: field ( `field.*`, 15) in 03, 04 (3), 05 (1), 06 ( `metadata.*`, 22), 06a (2), 07 (3), 07a ( `content.*`, 2), 08 (6), 09 (8), 10 (1). Heaviest single rules: `duplicate.cross_case_match` (40),

`copy_move.cluster_detected` / `cross_doc.amount_mismatch` / `signature.reused_different_signer` (35). Because weights sum, a case with an amount mismatch (35) plus a duplicate page (40) reaches 75 → **high**.

## Unscored findings and the coverage warning

Medium/high findings that **no rule covers** are logged as warnings ("No risk rule covers ...") so a new finding type cannot silently go unscored. `metadata_forensics_correlation` and `experimental_signal_notice` are excluded on purpose (see 08).

## Rule templates — how a new company gets its rules

---

- `risk_rule_templates` is a platform-level table (no `company_id` ; only the platform database role can reach it). Platform admins edit it under **Platform > Rule templates** ( `/platform/risk-rule-templates` ): weight, severity, active flag, or a new rule. Each template edit is a platform audit event ( `risk_rule_template_created/updated` , old → new values).
- Creating a company ( `POST /platform/companies` ) calls `seed_risk_rules(db, company_id)` , which copies every **active** template row into that company's `risk_rules` as version 1, plus default thresholds (30 / 60) into `risk_settings` .
- From then on the copy is the company's own. **Editing the template never changes an existing company** — only companies created afterwards. This is the same "no retroactive change" principle as rule versioning below.
- The template was seeded from the Default Company's current rules (38).

## Versioning — history is never rewritten

---

- **Rules are immutable.** `PATCH /settings/risk-rules/{rule_id}` (weight, severity, `is_active` ) INSERTs a new row with `version + 1` in the company's rules; the old row is untouched (the database grants only `SELECT + INSERT` on `risk_rules` ). The current rule is the highest version — there is no mutable "is\_current" flag.
- **Creating a rule** ( `POST /settings/risk-rules` ) inserts version 1; it applies to cases scored from then on. The editor (a Reviewer L2) picks a match kind and parameters from the catalog ( `GET /settings/risk-rule-options` ); nobody writes JSON, and a choice the engine cannot evaluate is a 422. Wording ( `reason_template` ) and `condition` cannot be edited after creation.
- **Every assessment freezes what it used:** the rendered reason text, and `risk_rules_version_snapshot = {"rules": [{rule_pk, rule_id, version, weight, severity}, ...], "thresholds": {"medium": ..., "high": ...}}` . Retuning weights or thresholds later cannot change a historical case's score or reasons.

## Output shape

---

**Stored** — `case_risk_assessments` (insert-only), the full record:

```
{
  "id": "uuid", "case_id": "uuid", "computed_at": "2026-09-20T17:09:12Z",
  "score": 75, "raw_score": 75.0, "tier": "high",
  "triggered_reasons": [
    // sorted by weight desc, then rule_id
    { "rule_id": "duplicate.cross_case_match", "rule_version": 1, "category": "duplication",
      "check_type": "duplicate_detection", "severity": "high", "weight": 40.0,
      "reason": "Page 1 of 'Invoice.pdf' is a near-identical match ... to page 1 of 'X.pdf' in case CASE-....",
      "document_id": "uuid", "document_filename": "Invoice.pdf"}, // both null for case-level rules
    { "rule_id": "cross_doc.amount_mismatch", "rule_version": 1, "weight": 35.0, "...": "..." }
  ],
  "risk_rules_version_snapshot": {
    // exactly which immutable versions produced the score
    "rules": [{ "rule_pk": "uuid", "rule_id": "duplicate.cross_case_match", "version": 1, "weight": 40.0,
      "severity": "high"}, "..."],
    "thresholds": { "medium": 30, "high": 60 }
  },
  "evidence_fingerprint": "9f2c..." // SHA-256 used to skip unchanged re-scores
}
```

**Exposed by the API** — `RiskAssessmentSchema` ON `GET /cases/{id}` ( `assessment` ; reviewers and platform admins only, `null` for a submitter) is a **subset**: `{id, score, raw_score, tier, computed_at, triggered_reasons[], thresholds}`, where each reason is a `RiskReasonSchema` with the fields shown above and `thresholds` is copied from the snapshot. The rule-version snapshot and the evidence fingerprint stay internal (the snapshot is printed in the report's Section 6, the fingerprint is never shown).

## What the tier does

| Tier          | Effect in the code                                                                                            |
|---------------|---------------------------------------------------------------------------------------------------------------|
| any           | Case flag/badge colour; case moves to <code>pending_manual_review</code> ; report Executive Summary           |
| low           | Reviewer may approve without a written justification                                                          |
| medium / high | Approving needs a justification of $\geq 10$ characters (422 otherwise); rejecting/escalating always possible |
| —             | <b>No</b> auto-approval, <b>no</b> automatic escalation to L2, <b>no</b> SLA (none of these exist)            |

Submitters see only a neutral "In review" flag, never the tier, score or reasons — so a bad actor cannot learn what to evade.

## Known limitations

- **Weights are judgment, not calibration.** The seeded values were chosen to be sensible, not fitted to data; tune them against real cases (Settings > Risk Rules, per company; or the platform template for future companies). The thresholds default to 30/60.
- Rules only see what checks store. A check that fails (status `failed` ) contributes nothing, and the case still scores — a `failed` forensic task under-reports risk silently (the failure is in the audit log and shown as a failed check, but no rule fires on it).
- Additive scoring: many weak signals can outweigh one strong one, and signals are not de-correlated (an edited PDF may trip metadata, ELA and visual rules for one root cause).
- The default rules encode this deployment's judgment about weak signals (ELA, AI-generation and signature checks are deliberately light).
- A case re-scores when evidence changes (for example a signature comparison finishing), so the score a reviewer first saw can change while they are looking at the case.